🔬 WiFi CSI Fall Detection System: A Deep Learning Approach

Project Overview

This project presents an advanced, AI-powered system for real-time fall detection utilizing **WiFi Channel State Information (CSI)**. Leveraging the pervasive nature of WiFi signals, this system offers a non-intrusive and privacy-preserving solution for monitoring human activities, particularly focusing on identifying fall events in vulnerable populations. The core innovation lies in the application of sophisticated deep learning architectures, specifically **ResNet1D** and a **Hybrid Convolutional Neural Network (CNN) with Bidirectional Long Short-Term Memory (BiLSTM)**, to analyze subtle disturbances in WiFi signals caused by human motion.

Developed with a strong emphasis on academic rigor and practical applicability, this system aims to contribute to the field of ambient assisted living and smart healthcare technologies. The robust design and comprehensive evaluation demonstrate a high standard of engineering and scientific methodology.

🎯 Key Features

- **Real-time CSI Acquisition**: Utilizes **ESP32 microcontrollers** configured as both a **transmitter (Tx)** and a **receiver (Rx)** to capture raw CSI data streams. This setup enables precise measurement of WiFi signal amplitude and phase variations caused by environmental changes, including human movement.
- **Advanced Deep Learning Models**: Implements and evaluates two distinct deep learning models:
  - **ResNet1D**: A one-dimensional Residual Network optimized for processing sequential CSI time-series data, capturing intricate temporal features indicative of fall patterns.
  - **Hybrid CNN-BiLSTM**: A combined architecture that leverages CNNs for spatial feature extraction from CSI subcarriers and BiLSTMs for robust temporal dependency modeling, enhancing the system's ability to distinguish falls from normal activities.
- **Automated Alert System**: Integrates with `pywhatkit` to provide instant **WhatsApp notifications** upon fall detection, ensuring timely intervention in emergency scenarios.
- **Comprehensive Data Management**: Includes functionalities for real-time CSI data collection, preprocessing, and structured storage, facilitating continuous model

improvement and validation.

- **Interactive User Interface**: A **Tkinter-based Graphical User Interface (GUI)** for live monitoring of CSI signals, real-time fall detection confidence, and system status, enhancing usability and operational oversight.
- **Robust Signal Preprocessing**: Incorporates techniques such as Exponential Moving Average (EMA) filtering and signal centering to mitigate noise and environmental interference, ensuring high-quality input for the deep learning models.

🛠️ Technical Stack & Methodologies

Hardware Implementation

The system's foundation is built upon two **ESP32 microcontrollers**. One ESP32 acts as a **transmitter**, continuously emitting WiFi beacon frames. The second ESP32 functions as a **receiver**, capturing the CSI data from these frames. Custom firmware on the ESP32s enables the extraction of raw CSI measurements, which are then transmitted via a serial interface (e.g., USB-to-serial converter) to a host computer for processing. This low-cost, ubiquitous hardware setup makes the solution scalable and accessible.

Software Components

- **Programming Language**: Python 3.x
- **Deep Learning Frameworks**: **TensorFlow** and **Keras** for model definition, training, and inference.
- **Data Science Libraries**: **NumPy** and **Pandas** for efficient data manipulation, numerical operations, and signal processing.
- **Machine Learning Utilities**: **Scikit-learn** for data preprocessing (e.g., `StandardScaler`), model evaluation (e.g., `classification_report`, `confusion_matrix`, `roc_auc_score`, `f1_score`), and utility functions like `compute_class_weight` for handling imbalanced datasets.
- **Visualization**: **Matplotlib** and **Seaborn** for generating insightful plots, including real-time signal visualizations, confusion matrices, and performance curves.
- **User Interface**: **Tkinter** for developing the interactive GUI, providing a user-friendly interface for system control and monitoring.
- **Serial Communication**: **PySerial** for establishing and managing the communication link between the host computer and the ESP32 CSI receiver.
- **Notification Service**: **PyWhatKit** for programmatic interaction with WhatsApp, enabling automated fall alerts.

Deep Learning Architectures

ResNet1D

The ResNet1D model is designed to effectively learn features from one-dimensional time-series data. Its architecture incorporates residual blocks, which help in training very deep networks by allowing gradients to flow more easily through the network. This prevents vanishing gradient problems and enables the model to capture long-range dependencies and subtle patterns within the CSI amplitude variations over time.

Hybrid CNN-BiLSTM

This hybrid model combines the strengths of Convolutional Neural Networks (CNNs) and Bidirectional Long Short-Term Memory (BiLSTMs). The CNN layers are initially used to extract local spatial features across the CSI subcarriers, effectively identifying patterns within the frequency domain. The output of the CNN layers is then fed into BiLSTM layers, which are particularly adept at learning long-term temporal dependencies in sequential data. The bidirectional nature of BiLSTMs allows the model to process sequences in both forward and backward directions, providing a more comprehensive understanding of the temporal context of CSI signals leading up to and during a fall event.

📂 Project Structure

- `final model.py` : Contains the primary script for training and evaluating the ResNet1D deep learning model, including data loading, preprocessing pipelines, model definition, training loops, and performance metric calculation.
- `iDcnn+biLSTM live/` :
  - `app.py` : The main application script for the real-time fall detection system, featuring the Tkinter GUI, serial data acquisition, signal processing, and live inference using the trained deep learning models.
  - `model.py` : Defines the architecture for the Hybrid CNN-BiLSTM model.
- `MODELS/` : Stores pre-trained deep learning models (`.h5` , `.keras` , `.tflite`), trained `StandardScaler` objects (`.pkl`), and various evaluation artifacts such as confusion matrices and model comparison plots.
- `RAW/` , `TEST/` , `NEW_ROOM_DATA/` : Directories housing the collected CSI datasets, categorized into 'fall' and 'no fall' events, used for model training, testing, and validation. These datasets are critical for developing and evaluating the robustness of the fall detection algorithms.

- `message.py` : Manages the alert and notification functionalities, specifically the integration with `pywhatkit` for sending WhatsApp messages.

⚙️ Setup & Installation

1. **Clone the repository**:

```
Bash
```

```
git clone https://github.com/joseph-m-Benny/wifi-csi-fall-detection.git
cd wifi-csi-fall-detection
```

2. **Install Python Dependencies**:It is highly recommended to use a virtual environment.

```
Bash
```

```
pip install tensorflow scikit-learn pandas numpy matplotlib pyserial pywhatk
```

3. **Hardware Configuration (ESP32)**:

- **Firmware**: Ensure your ESP32 devices are flashed with appropriate CSI capture firmware (e.g., ESP32-CSI-Tool or similar projects).
- **Connection**: Connect the ESP32 receiver to your host computer via USB. Identify the assigned serial port.
- **Configuration**: Update the `_PORT` variable within `iDcnn+biLSTM live/app.py` and `message.py` to match the serial port of your ESP32 receiver (e.g., `COM5` on Windows, `/dev/ttyUSB0` on Linux/macOS).

🖥️ Usage

- **Model Training**: To train the ResNet1D model with your datasets:

```
Bash
```

```
python "final model.py"
```

- **Live Fall Detection System**: To launch the real-time GUI application for live monitoring and detection:

```
Bash
```

```
python "iDcnn+biLSTM live/app.py"
```

📊 Experimental Results & Discussion

The system demonstrates high efficacy in distinguishing fall events from normal activities. Performance metrics, including accuracy, precision, recall, F1-score, and ROC AUC, are rigorously evaluated. The use of both ResNet1D and the Hybrid CNN-BiLSTM models allows for comparative analysis, highlighting the strengths of each architecture in processing complex CSI data. Detailed confusion matrices and performance reports are generated and stored in the `MODELS/` directory, providing transparent insights into the model's behavior and reliability.

This project underscores the potential of WiFi CSI as a robust and privacy-friendly sensing modality for critical applications like fall detection, paving the way for future advancements in smart home and healthcare systems.

Developed as part of a WiFi CSI Mini Project.